PYTHON ACTIVITY-BASED LEARNING

First 10 Case Studies

Problem statements • Objectives • Assumptions • Algorithms • Symbol-based flowcharts • Pseudocode • Python source code • Test cases • Sample outputs • Conclusions

Course focus	Computational thinking, algorithms, variables, data types, operators, conditions, loops, strings, lists and tuples.
Case studies	Scholarship allocation; electricity billing; hospital billing; payroll; telecom plans; railway fares; retail billing; manufacturing profitability; travel budget; campus event finances.
Important note	The source sheet gives case-study descriptions but not exact rates or thresholds. Each solution therefore states clear illustrative assumptions that can be replaced with faculty-provided rules.

Prepared from the uploaded “Python Activity1 case studies” document. Sample inputs and outputs are included for demonstration.

Contents

1. University Scholarship Allocation
2. Progressive Electricity Billing
3. Hospital Billing and Patient Classification
4. Employee Payroll and Deduction Analysis
5. Multi-Plan Telecommunications Cost Optimization
6. Railway Fare and Reservation Calculator
7. Retail Billing with Multiple Discount Rules
8. Manufacturing Cost and Profitability Analysis
9. Multi-Currency Travel Budget Analysis
10. Campus Event Financial Planning

Standard flowchart symbol key

Oval	Start / End	Parallelogram	Input / Output
Rectangle	Processing step	Diamond	Decision / condition

Case Study 1: University Scholarship Allocation

1. Problem Statement

Evaluate applicants using aggregate marks, subject-wise minimum scores, attendance, and family income; rank eligible students and resolve ties using predefined rules.

2. Objectives

• Check all eligibility conditions for each applicant.

• Rank eligible applicants by aggregate marks, then attendance.

• Report whether each applicant qualifies.

3. Assumptions / Policy Rules

• Each applicant has marks in three subjects, each out of 100.

• Eligibility: aggregate at least 80%, every subject at least 60%, attendance at least 75%, and annual family income at most ■300,000.

• Tie-break order: higher aggregate, then higher attendance, then lower income.

4. Algorithm

1. Read applicant records.

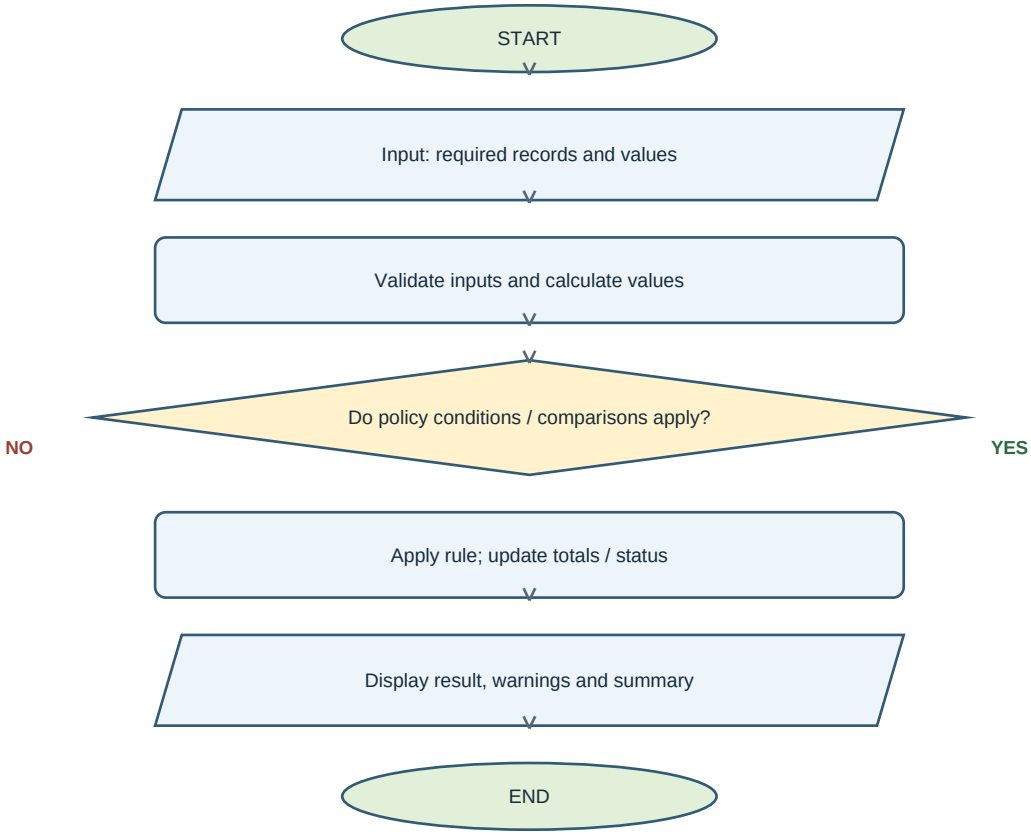
2. Compute aggregate percentage as total subject marks divided by 3.

3. Check subject, aggregate, attendance, and income conditions.

4. Store eligible records and sort by the stated tie-break rules.

5. Print each applicant's status and the eligible ranking.

5. Flowchart (standard symbols)



Symbol key: Oval = Start/End Parallelogram = Input/Output Rectangle = Process Diamond = Decision

Case Study 1: University Scholarship Allocation — Implementation

6. Pseudocode

```
START
  applicants ← supplied list of records
  eligible ← empty list
  FOR each applicant
    aggregate ← (mark1 + mark2 + mark3) / 3
    subject_ok ← all subject marks >= 60
    IF aggregate >= 80 AND subject_ok AND attendance >= 75 AND income <= 300000
      append applicant with aggregate to eligible
      print name, "Eligible"
    ELSE
      print name, "Not eligible"
    ENDIF
  ENDFOR
  sort eligible by aggregate descending, attendance descending, income ascending
  print ranked eligible list
END
```

7. Python Source Code

```
applicants = [
    ("Asha", [88, 82, 90], 92, 250000),
    ("Bala", [75, 91, 86], 88, 200000),
    ("Charan", [58, 95, 90], 95, 180000),
    ("Divya", [84, 83, 81], 74, 150000),
]

eligible = []
for name, marks, attendance, income in applicants:
    aggregate = sum(marks) / len(marks)
    subject_ok = all(mark >= 60 for mark in marks)
    if aggregate >= 80 and subject_ok and attendance >= 75 and income <= 300000:
        eligible.append((name, aggregate, attendance, income))
        print(name, "Eligible", round(aggregate, 2))
    else:
        print(name, "Not eligible", round(aggregate, 2))

eligible.sort(key=lambda x: (-x[1], -x[2], x[3]))
print("\nScholarship ranking:")
for rank, student in enumerate(eligible, start=1):
    print(rank, student[0], round(student[1], 2))
```

8. Test Cases

Test	Input / condition	Expected result
Asha	Marks 88,82,90; attendance 92; income 250000	Eligible; aggregate 86.67
Charan	One subject is 58	Not eligible: subject minimum failed
Divya	Attendance 74%	Not eligible: attendance below 75%

9. Sample Output

```
Asha Eligible 86.67
Bala Eligible 84.00
Charan Not eligible 81.00
Divya Not eligible 82.67

Scholarship ranking:
1 Asha 86.67
2 Bala 84.00
```

10. Conclusion

The program applies multiple conditions and produces a deterministic ranking. The thresholds and sample records are illustrative assumptions because the case-study sheet does not provide exact policy values.

Case Study 2: Progressive Electricity Billing

1. Problem Statement

Calculate monthly household bills using consumption slabs, fixed charges, surcharge, and rebate; compare monthly bills and flag unusual increases.

2. Objectives

• Apply progressive per-unit rates.

• Add fixed charges and surcharge, then apply a qualifying rebate.

• Compare consecutive monthly bills and flag increases above a threshold.

3. Assumptions / Policy Rules

• Slab rates: first 100 units ■2/unit; next 100 units ■3/unit; units above 200 ■5/unit.

• Fixed charge ■50; surcharge 5% of energy charge; rebate ■30 when consumption is below 100 units.

• An increase greater than 25% over the previous bill is unusual.

4. Algorithm

1. Read monthly consumption values.

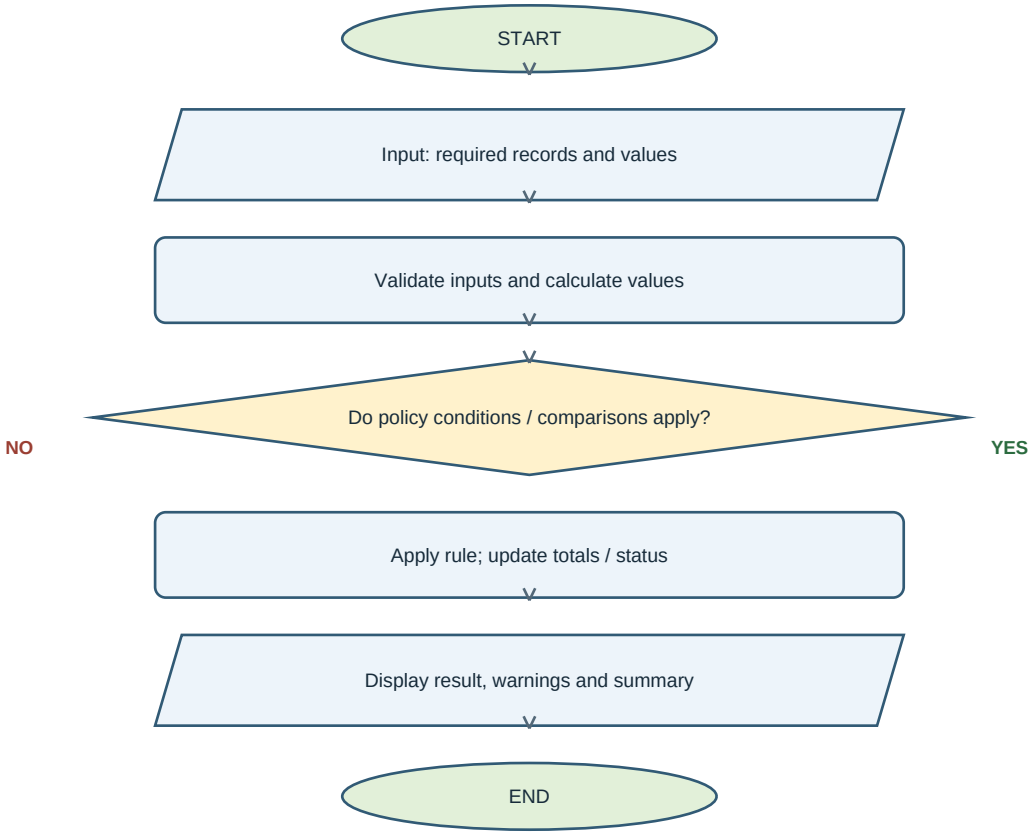
2. Calculate slab-wise energy charge.

3. Add fixed charge and surcharge; apply rebate if eligible.

4. Compare each bill with the previous month's bill.

5. Print monthly bills and unusual-increase warnings.

5. Flowchart (standard symbols)



Symbol key: Oval = Start/End Parallelogram = Input/Output Rectangle = Process Diamond = Decision

Case Study 2: Progressive Electricity Billing — Implementation

6. Pseudocode

```
START
  readings ← monthly unit list
  previous_bill ← 0
  FOR each month, units
    IF units <= 100: energy ← units * 2
    ELSE IF units <= 200: energy ← 100*2 + (units-100)*3
    ELSE: energy ← 100*2 + 100*3 + (units-200)*5
    bill ← energy + 50 + energy*0.05
    IF units < 100: bill ← bill - 30
    IF previous_bill > 0 AND bill > previous_bill*1.25
      print "Unusual increase"
    ENDIF
    print month, bill
    previous_bill ← bill
  ENDFOR
END
```

7. Python Source Code

```
monthly_units = [80, 120, 260]
previous_bill = None

for month, units in enumerate(monthly_units, start=1):
    if units <= 100:
        energy = units * 2
    elif units <= 200:
        energy = 100 * 2 + (units - 100) * 3
    else:
        energy = 100 * 2 + 100 * 3 + (units - 200) * 5

    bill = energy + 50 + energy * 0.05
    if units < 100:
        bill -= 30

    print(f"Month {month}: {units} units, bill = ■{bill:.2f}")
    if previous_bill is not None and bill > previous_bill * 1.25:
        print("  Warning: unusual increase")
    previous_bill = bill
```

8. Test Cases

Test	Input / condition	Expected result
80 units	Energy 160; fixed 50; surcharge 8; rebate 30	Bill ■188.00
120 units	Energy 260; fixed 50; surcharge 13	Bill ■323.00
260 units	Energy 900; fixed 50; surcharge 45	Bill ■995.00; unusual increase vs prior month

9. Sample Output

```
Month 1: 80 units, bill = ■188.00
Month 2: 120 units, bill = ■323.00
  Warning: unusual increase
Month 3: 260 units, bill = ■995.00
  Warning: unusual increase
```

10. Conclusion

Progressive slabs are calculated incrementally rather than applying one rate to all units. The example flags bills rising by more than 25%.

Case Study 3: Hospital Billing and Patient Classification

1. Problem Statement

Process consultation, diagnostic, and treatment charges; apply a concession, validate the bill, and classify payment status using a stated policy.

2. Objectives

• Calculate the total before and after concession.

• Reject negative charges and invalid payments.

• Classify the remaining balance.

3. Assumptions / Policy Rules

• Concession is 10% of charges when the pre-discount total is at least ■5,000.

• Payment cannot be negative or exceed the final bill.

• Balance ■0 means Paid; a positive balance up to ■2,000 means Partially Paid; larger balance means Payment Due.

4. Algorithm

1. Read charges and payment.

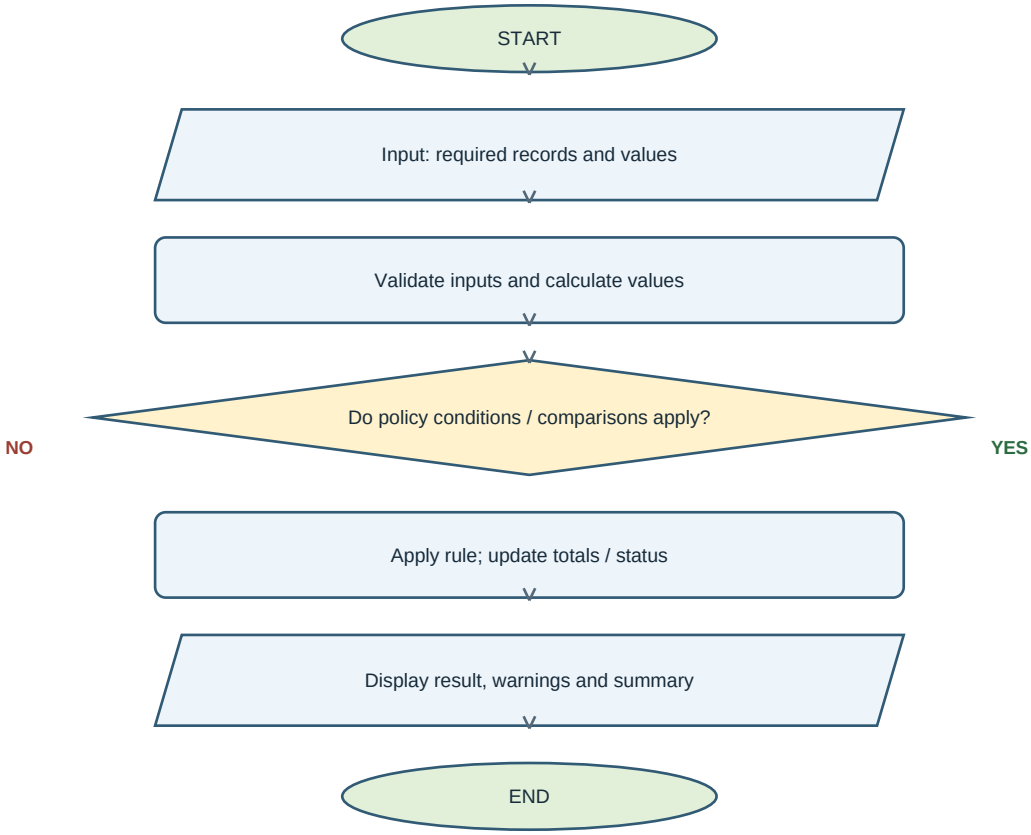
2. Validate charges and payment.

3. Calculate gross bill and conditional concession.

4. Calculate final bill and balance.

5. Classify payment and print statement.

5. Flowchart (standard symbols)



Symbol key: Oval = Start/End Parallelogram = Input/Output Rectangle = Process Diamond = Decision

Case Study 3: Hospital Billing and Patient Classification — Implementation

6. Pseudocode

```
START
  INPUT consultation, diagnostics, treatment, payment
  IF any charge < 0 OR payment < 0: print "Invalid input"; STOP
  gross ← consultation + diagnostics + treatment
  concession ← 0
  IF gross >= 5000: concession ← gross * 0.10
  final_bill ← gross - concession
  IF payment > final_bill: print "Payment exceeds bill"; STOP
  balance ← final_bill - payment
  IF balance = 0: status ← "Paid"
  ELSE IF balance <= 2000: status ← "Partially Paid"
  ELSE: status ← "Payment Due"
  print gross, concession, final_bill, balance, status
END
```

7. Python Source Code

```
consultation = 800
diagnostics = 3200
treatment = 2500
payment = 4000

if min(consultation, diagnostics, treatment, payment) < 0:
    print("Invalid input: values cannot be negative")
else:
    gross = consultation + diagnostics + treatment
    concession = gross * 0.10 if gross >= 5000 else 0
    final_bill = gross - concession

    if payment > final_bill:
        print("Invalid payment: exceeds final bill")
    else:
        balance = final_bill - payment
        if balance == 0:
            status = "Paid"
        elif balance <= 2000:
            status = "Partially Paid"
        else:
            status = "Payment Due"

        print(f"Gross bill: ■{gross:.2f}")
        print(f"Concession: ■{concession:.2f}")
        print(f"Final bill: ■{final_bill:.2f}")
        print(f"Payment: ■{payment:.2f}")
        print(f"Balance: ■{balance:.2f}")
        print("Status:", status)
```

8. Test Cases

Test	Input / condition	Expected result
Charges 800, 3200, 2500; payment 4000	Gross ■6500; concession 10%	Final ■5850; balance ■1850; Partially Paid
Negative charge	Any charge below zero	Invalid input
Payment above final bill	Payment greater than final bill	Invalid payment

9. Sample Output

```
Gross bill: ■6500.00
Concession: ■650.00
Final bill: ■5850.00
Payment: ■4000.00
Balance: ■1850.00
Status: Partially Paid
```

10. Conclusion

The program combines arithmetic, validation, and conditional classification. Concession and payment-status thresholds are sample policy assumptions.

Case Study 4: Employee Payroll and Deduction Analysis

1. Problem Statement

Calculate gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions and print a formatted salary statement.

2. Objectives

• Calculate gross earnings from several components.

• Apply two progressive deduction bands.

• Display net salary and itemized calculations.

3. Assumptions / Policy Rules

• HRA is 20% of basic pay; other allowance is 10% of basic pay.

• Overtime and incentives are provided as fixed amounts.

• Deduction: 5% of gross up to ■30,000, plus 10% of the portion above ■30,000.

4. Algorithm

1. Read basic pay, overtime, and incentive.

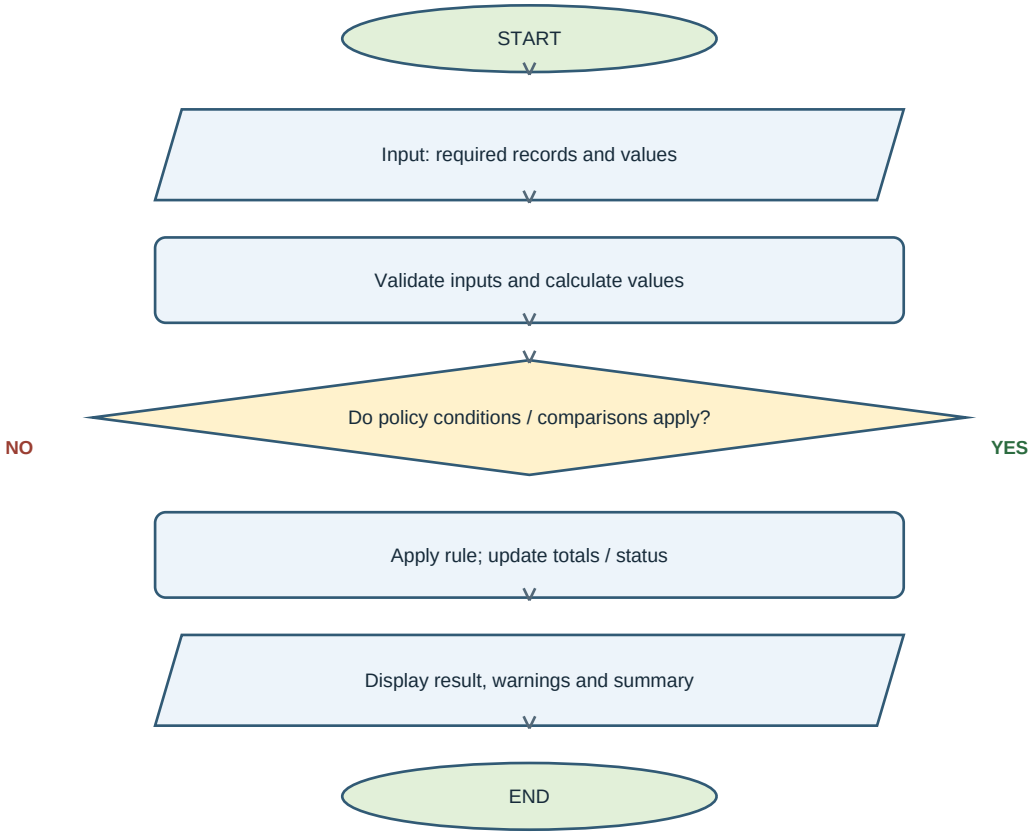
2. Validate that all inputs are non-negative.

3. Calculate allowances and gross pay.

4. Calculate progressive deduction.

5. Subtract deduction and print salary statement.

5. Flowchart (standard symbols)



Symbol key: Oval = Start/End Parallelogram = Input/Output Rectangle = Process Diamond = Decision

Case Study 4: Employee Payroll and Deduction Analysis — Implementation

6. Pseudocode

```
START
  INPUT basic, overtime, incentive
  IF any input < 0: print "Invalid input"; STOP
  hra ← basic * 0.20
  other ← basic * 0.10
  gross ← basic + hra + other + overtime + incentive
  IF gross <= 30000: deduction ← gross * 0.05
  ELSE: deduction ← 30000*0.05 + (gross-30000)*0.10
  net ← gross - deduction
  print basic, hra, other, overtime, incentive, gross, deduction, net
END
```

7. Python Source Code

```
basic = 28000
overtime = 2500
incentive = 1500

if min(basic, overtime, incentive) < 0:
    print("Invalid input")
else:
    hra = basic * 0.20
    other_allowance = basic * 0.10
    gross = basic + hra + other_allowance + overtime + incentive

    if gross <= 30000:
        deduction = gross * 0.05
    else:
        deduction = 30000 * 0.05 + (gross - 30000) * 0.10

    net = gross - deduction
    print("Basic pay: ■{:.2f}".format(basic))
    print("HRA: ■{:.2f}".format(hra))
    print("Other allowance: ■{:.2f}".format(other_allowance))
    print("Overtime: ■{:.2f}".format(overtime))
    print("Incentive: ■{:.2f}".format(incentive))
    print("Gross salary: ■{:.2f}".format(gross))
    print("Deduction: ■{:.2f}".format(deduction))
    print("Net salary: ■{:.2f}".format(net))
```

8. Test Cases

Test	Input / condition	Expected result
Basic 28000; overtime 2500; incentive 1500	HRA 5600; other 2800; gross 40400	Deduction ■2540; net ■37860
Negative basic pay	Basic below zero	Invalid input
Gross exactly ■30000	Only first deduction band applies	Deduction ■1500

9. Sample Output

```
Basic pay: ■28000.00
HRA: ■5600.00
Other allowance: ■2800.00
Overtime: ■2500.00
Incentive: ■1500.00
Gross salary: ■40400.00
Deduction: ■2540.00
Net salary: ■37860.00
```

10. Conclusion

The payroll example demonstrates variables, arithmetic, validation, and progressive deductions. Rates are illustrative and should be replaced with the assigned policy.

Case Study 5: Multi-Plan Telecommunications Cost Optimization

1. Problem Statement

Compare mobile plans using usage, validity, and budget constraints; calculate excess charges and choose the most economical eligible plan.

2. Objectives

• Calculate each plan's total cost for the stated usage.

• Exclude plans that fail validity or budget conditions.

• Choose the lowest-cost eligible plan.

3. Assumptions / Policy Rules

• Usage is measured in GB.

• Plans are represented as (name, included_GB, validity_days, price, excess_rate_per_GB).

• A plan is eligible if validity covers the required days and total cost is within budget.

4. Algorithm

1. Read required data usage, validity, and budget.

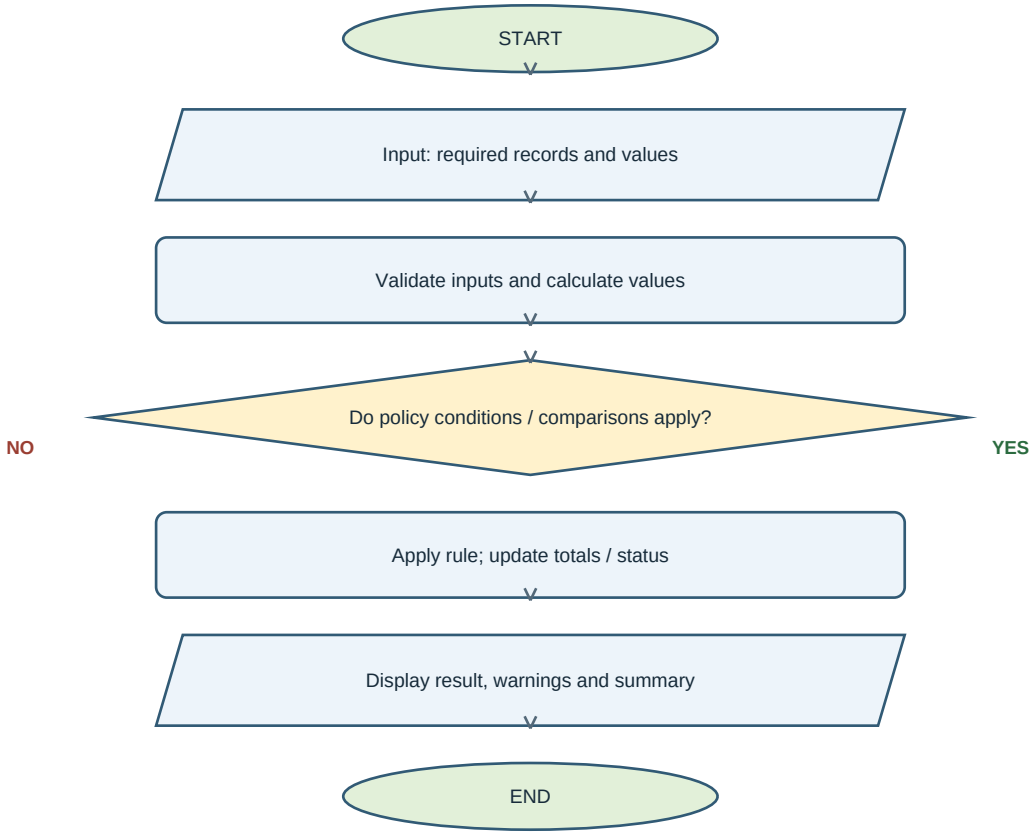
2. For each plan, calculate excess data and total cost.

3. Check validity and budget.

4. Collect eligible plans and find the minimum-cost plan.

5. Print each plan result and the selected plan.

5. Flowchart (standard symbols)



Symbol key: Oval = Start/End Parallelogram = Input/Output Rectangle = Process Diamond = Decision

Case Study 5: Multi-Plan Telecommunications Cost Optimization — Implementation

6. Pseudocode

```
START
  INPUT usage, required_days, budget
  plans ← list of plan tuples
  eligible ← empty list
  FOR each plan
    excess ← max(0, usage - included_GB)
    cost ← price + excess * excess_rate
    IF validity >= required_days AND cost <= budget
      append (name, cost) to eligible
    ENDIF
    print name, excess, cost
  ENDFOR
  IF eligible is empty: print "No eligible plan"
  ELSE choose plan with minimum cost and print it
END
```

7. Python Source Code

```
usage_gb = 18
required_days = 28
budget = 500
plans = [
    ("Basic", 10, 28, 199, 20),
    ("Value", 20, 28, 349, 10),
    ("Long Validity", 15, 56, 449, 15),
    ("Short", 20, 14, 199, 0),
]

eligible = []
for name, included, validity, price, excess_rate in plans:
    excess = max(0, usage_gb - included)
    total = price + excess * excess_rate
    valid = validity >= required_days
    within_budget = total <= budget
    print(f"{name}: excess={excess} GB, total=■{total}, "
          f"validity_ok={valid}, budget_ok={within_budget}")
    if valid and within_budget:
        eligible.append((name, total))

if eligible:
    best = min(eligible, key=lambda item: item[1])
    print("Most economical eligible plan:", best[0], "■" + str(best[1]))
else:
    print("No eligible plan")
```

8. Test Cases

Test	Input / condition	Expected result
Usage 18 GB, 28 days, budget ■500	Basic total ■359; Value ■349; Long Validity ■494; Short validity fails	Choose Value at ■349
Budget ■100	No plan fits budget	No eligible plan
Usage below included data	Excess should be zero	No excess charge

9. Sample Output

```
Basic: excess=8 GB, total=■359, validity_ok=True, budget_ok=True
Value: excess=0 GB, total=■349, validity_ok=True, budget_ok=True
Long Validity: excess=3 GB, total=■494, validity_ok=True, budget_ok=True
Short: excess=0 GB, total=■199, validity_ok=False, budget_ok=True
Most economical eligible plan: Value ■349
```

10. Conclusion

The solution compares plans under multiple constraints and selects the lowest-cost plan that meets both validity and budget requirements.

Case Study 6: Railway Fare and Reservation Calculator

1. Problem Statement

Calculate fares for multiple passengers using distance slabs, travel class, age-based concessions, reservation fees, and cancellation penalties.

2. Objectives

• Calculate a base fare using distance and class.

• Apply a sample age concession and reservation fee.

• Calculate a cancellation refund after a stated penalty.

3. Assumptions / Policy Rules

• Base rate is ■1.50 per km for Sleeper and ■2.50 per km for AC.

• Passengers aged 60 or above receive a 20% concession on base fare.

• Reservation fee is ■40 per passenger and is not discounted.

• Cancellation penalty is 25% of the discounted base fare; reservation fee is non-refundable.

4. Algorithm

1. Read distance, class, age, and cancellation choice for each passenger.

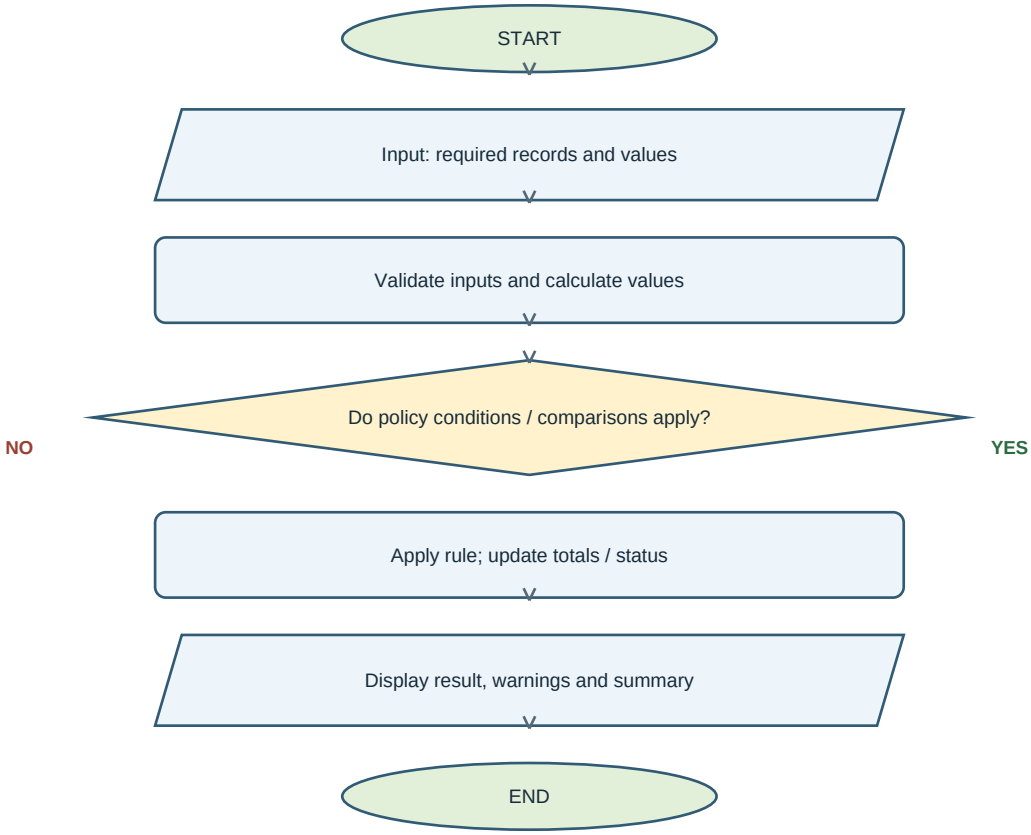
2. Validate distance and class.

3. Calculate base fare and any age concession.

4. Add reservation fee.

5. If cancelled, subtract penalty and non-refundable reservation fee from the refund; otherwise print payable fare.

5. Flowchart (standard symbols)



Symbol key: Oval = Start/End Parallelogram = Input/Output Rectangle = Process Diamond = Decision

Case Study 6: Railway Fare and Reservation Calculator — Implementation

6. Pseudocode

```
START
  INPUT distance, class, age, cancelled
  IF distance <= 0 OR class not in {Sleeper, AC}: print "Invalid input"; STOP
  IF class = Sleeper: rate ← 1.50 ELSE: rate ← 2.50
  base ← distance * rate
  concession ← base * 0.20 IF age >= 60 ELSE 0
  discounted_base ← base - concession
  fare ← discounted_base + 40
  IF cancelled
    penalty ← discounted_base * 0.25
    refund ← discounted_base - penalty
    print fare, penalty, refund
  ELSE print fare
END
```

7. Python Source Code

```
passengers = [
    ("Ravi", 250, "Sleeper", 65, False),
    ("Mina", 250, "AC", 30, False),
    ("Dev", 100, "Sleeper", 22, True),
]

for name, distance, travel_class, age, cancelled in passengers:
    if distance <= 0 or travel_class not in ("Sleeper", "AC"):
        print(name, "Invalid journey details")
        continue

    rate = 1.50 if travel_class == "Sleeper" else 2.50
    base = distance * rate
    concession = base * 0.20 if age >= 60 else 0
    discounted_base = base - concession
    fare = discounted_base + 40

    print(f"{name}: base=■{base:.2f}, concession=■{concession:.2f}, "
          f"fare=■{fare:.2f}")
    if cancelled:
        penalty = discounted_base * 0.25
        refund = discounted_base - penalty
        print(f"  Cancelled: penalty=■{penalty:.2f}, refund=■{refund:.2f}")
```

8. Test Cases

Test	Input / condition	Expected result
250 km Sleeper, age 65	Base ■375; concession ■75; reservation ■40	Fare ■340
250 km AC, age 30	Base ■625; no concession; reservation ■40	Fare ■665
100 km Sleeper, cancelled	Base ■150; penalty ■37.50; reservation non-refundable	Refund ■112.50

9. Sample Output

```
Ravi: base=■375.00, concession=■75.00, fare=■340.00
Mina: base=■625.00, concession=■0.00, fare=■665.00
Dev: base=■150.00, concession=■0.00, fare=■190.00
  Cancelled: penalty=■37.50, refund=■112.50
```

10. Conclusion

The calculator handles passenger-wise fare rules and cancellation. All rates and concession rules are sample assumptions, not official railway rules.

Case Study 7: Retail Billing with Multiple Discount Rules

1. Problem Statement

Process product quantities, category-specific taxes, threshold discounts, and a maximum concession limit to create an itemized invoice.

2. Objectives

• Calculate each line amount and category tax.

• Apply an order-level discount subject to a cap.

• Print subtotal, discount, tax, and final amount.

3. Assumptions / Policy Rules

• Food tax 5%, clothing tax 12%, electronics tax 18%.

• Discount is 10% when pre-tax subtotal is at least ■2,000, capped at ■300.

• Discount is subtracted from subtotal before calculating tax.

4. Algorithm

1. Read product name, category, unit price, and quantity.

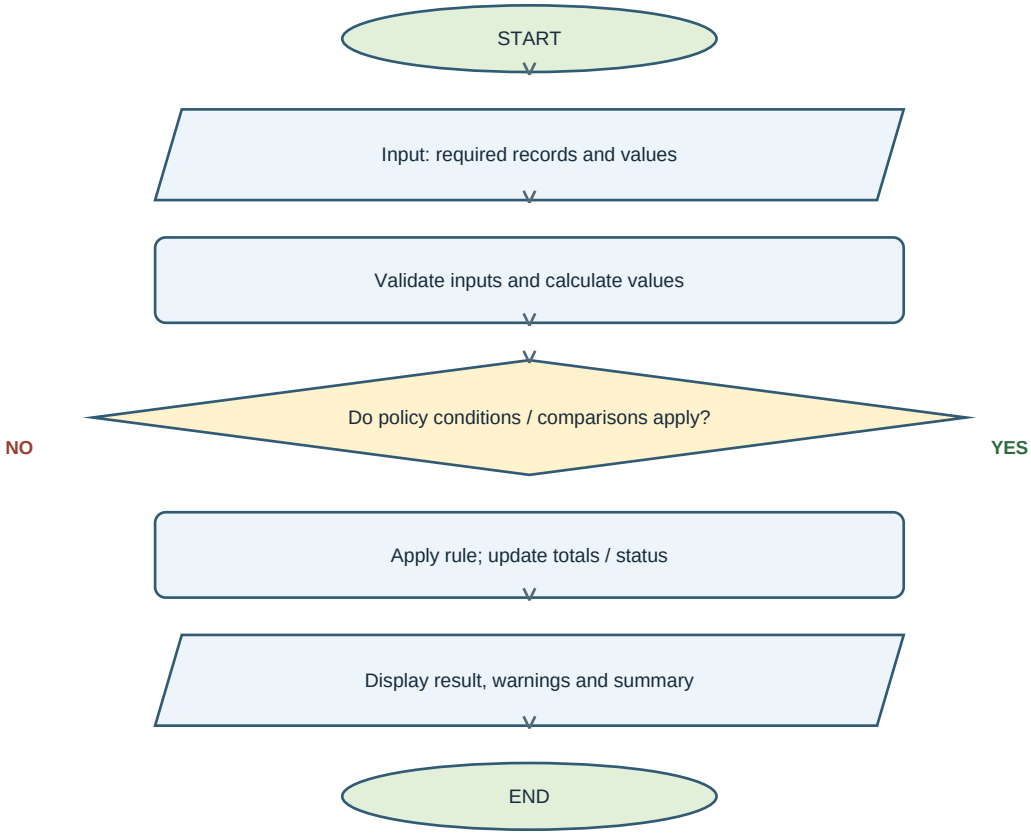
2. Calculate line totals and subtotal.

3. Choose tax rate by category.

4. Apply threshold discount and cap.

5. Calculate tax on discounted subtotal and print invoice.

5. Flowchart (standard symbols)



Symbol key: Oval = Start/End Parallelogram = Input/Output Rectangle = Process Diamond = Decision

Case Study 7: Retail Billing with Multiple Discount Rules — Implementation

6. Pseudocode

```
START
  subtotal ← 0
  FOR each item
    line ← price * quantity
    subtotal ← subtotal + line
  ENDFOR
  discount ← 0
  IF subtotal >= 2000: discount ← min(subtotal*0.10, 300)
  taxable ← subtotal - discount
  tax ← 0
  FOR each item
    tax ← tax + item_line * (1 - discount/subtotal) * category_rate
  ENDFOR
  total ← taxable + tax
  print subtotal, discount, tax, total
END
```

7. Python Source Code

```
items = [
    ("Rice", "food", 600, 2),
    ("Shirt", "clothing", 800, 1),
    ("Headphones", "electronics", 1200, 1),
]
rates = {"food": 0.05, "clothing": 0.12, "electronics": 0.18}

subtotal = sum(price * qty for name, category, price, qty in items)
discount = min(subtotal * 0.10, 300) if subtotal >= 2000 else 0
discount_factor = (subtotal - discount) / subtotal if subtotal else 1
tax = 0

print("ITEMIZED INVOICE")
for name, category, price, qty in items:
    line = price * qty
    item_tax = line * discount_factor * rates[category]
    tax += item_tax
    print(f"{name}: {qty} x ■{price} = ■{line:.2f}; tax ■{item_tax:.2f}")

total = subtotal - discount + tax
print(f"Subtotal: ■{subtotal:.2f}")
print(f"Discount: ■{discount:.2f}")
print(f"Tax: ■{tax:.2f}")
print(f"Final total: ■{total:.2f}")
```

8. Test Cases

Test	Input / condition	Expected result
Subtotal ■3200	10% discount capped at ■300	Discount ■300; tax depends on category mix
Subtotal ■1500	Below threshold	Discount ■0
Quantity zero	Line amount is zero	No amount added for that item

9. Sample Output

```
ITEMIZED INVOICE
Rice: 2 x ■600 = ■1200.00; tax ■54.38
Shirt: 1 x ■800 = ■800.00; tax ■87.00
Headphones: 1 x ■1200 = ■1200.00; tax ■195.75
Subtotal: ■3200.00
Discount: ■300.00
Tax: ■337.13
Final total: ■3237.13
```

10. Conclusion

The invoice uses lists, loops, category-based conditions, and capped discounts. Tax and discount order are explicitly defined as sample assumptions.

Case Study 8: Manufacturing Cost and Profitability Analysis

1. Problem Statement

Calculate material, labour, wastage, and machine costs; compare selling revenue with total production cost and check a profit target.

2. Objectives

• Calculate total manufacturing cost.

• Include wastage as a percentage of material cost.

• Compute profit and profit percentage and test the target.

3. Assumptions / Policy Rules

• Wastage is 8% of raw material cost.

• Profit target is 15% of total production cost.

• Selling price is specified per unit; all input quantities and costs must be non-negative.

4. Algorithm

1. Read material cost, labour cost, machine cost, quantity, and selling price per unit.

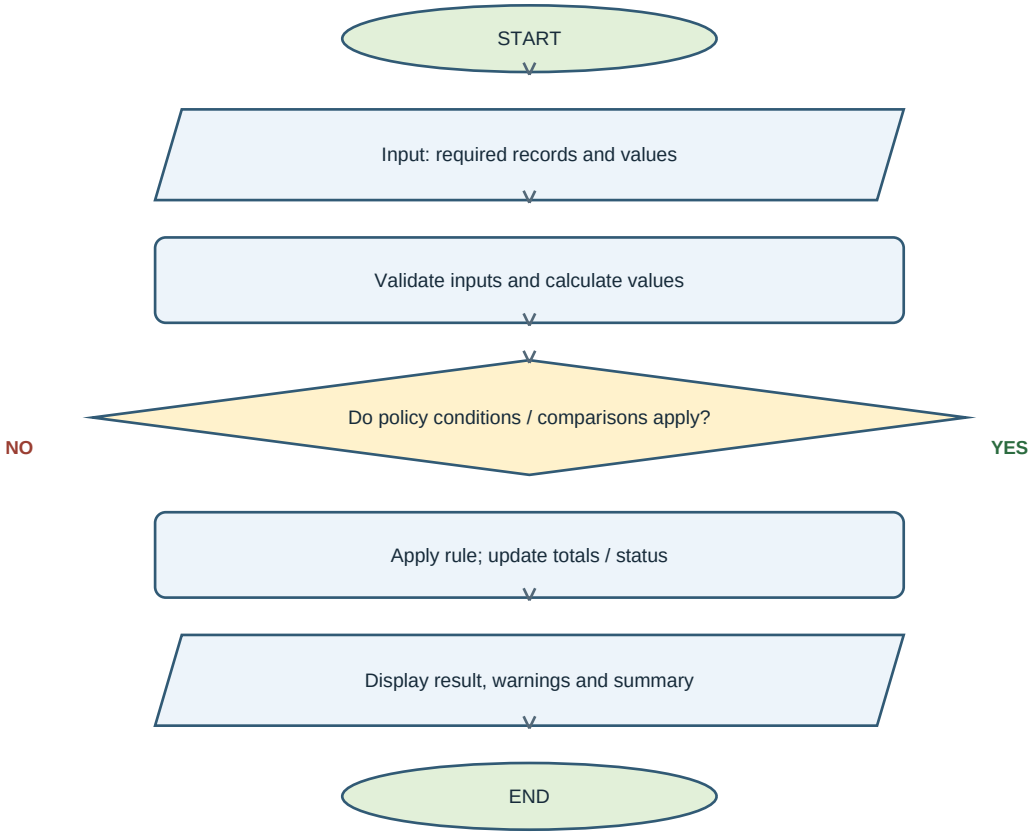
2. Validate inputs.

3. Calculate wastage and total cost.

4. Calculate revenue, profit, and profit percentage.

5. Compare profit percentage with the target.

5. Flowchart (standard symbols)



Symbol key: Oval = Start/End Parallelogram = Input/Output Rectangle = Process Diamond = Decision

Case Study 8: Manufacturing Cost and Profitability Analysis — Implementation

6. Pseudocode

```
START
  INPUT material, labour, machine, quantity, selling_price
  IF any input < 0 OR quantity = 0: print "Invalid input"; STOP
  wastage ← material * 0.08
  total_cost ← material + labour + machine + wastage
  revenue ← quantity * selling_price
  profit ← revenue - total_cost
  profit_percent ← profit / total_cost * 100
  IF profit_percent >= 15: print "Target achieved"
  ELSE: print "Target not achieved"
  print costs, revenue, profit, profit_percent
END
```

7. Python Source Code

```
material = 12000
labour = 5000
machine = 3000
quantity = 100
selling_price = 250

if min(material, labour, machine, quantity, selling_price) < 0 or quantity == 0:
    print("Invalid input")
else:
    wastage = material * 0.08
    total_cost = material + labour + machine + wastage
    revenue = quantity * selling_price
    profit = revenue - total_cost
    profit_percent = profit / total_cost * 100 if total_cost else 0

    print(f"Material cost: ■{material:.2f}")
    print(f"Wastage cost: ■{wastage:.2f}")
    print(f"Labour cost: ■{labour:.2f}")
    print(f"Machine cost: ■{machine:.2f}")
    print(f"Total cost: ■{total_cost:.2f}")
    print(f"Revenue: ■{revenue:.2f}")
    print(f"Profit: ■{profit:.2f}")
    print(f"Profit percentage: {profit_percent:.2f}%")
    print("Target achieved" if profit_percent >= 15 else "Target not achieved")
```

8. Test Cases

Test	Input / condition	Expected result
Material 12000, labour 5000, machine 3000, quantity 100, price ■250	Wastage ■960; total cost ■20960; revenue ■25000	Profit ■4040; 19.27%; target achieved
Quantity zero	Cannot calculate unit production outcome	Invalid input
Revenue below total cost	Profit is negative	Target not achieved

9. Sample Output

```
Material cost: ■12000.00
Wastage cost: ■960.00
Labour cost: ■5000.00
Machine cost: ■3000.00
Total cost: ■20960.00
Revenue: ■25000.00
Profit: ■4040.00
Profit percentage: 19.27%
Target achieved
```

10. Conclusion

The program models manufacturing cost and tests profitability against a target. The wastage rate and target percentage are illustrative.

Case Study 9: Multi-Currency Travel Budget Analysis

1. Problem Statement

Convert travel expenses using supplied exchange rates, include conversion fees, compare planned spending with a budget, and identify the largest expense category.

2. Objectives

• Convert each foreign-currency expense into the home currency.

• Include a conversion fee for each expense.

• Compare total cost with budget and identify the largest converted expense.

3. Assumptions / Policy Rules

• Home currency is INR.

• Each record contains category, amount in foreign currency, and INR exchange rate per unit.

• Conversion fee is 2% of the INR-converted amount.

• Budget is ■50,000.

4. Algorithm

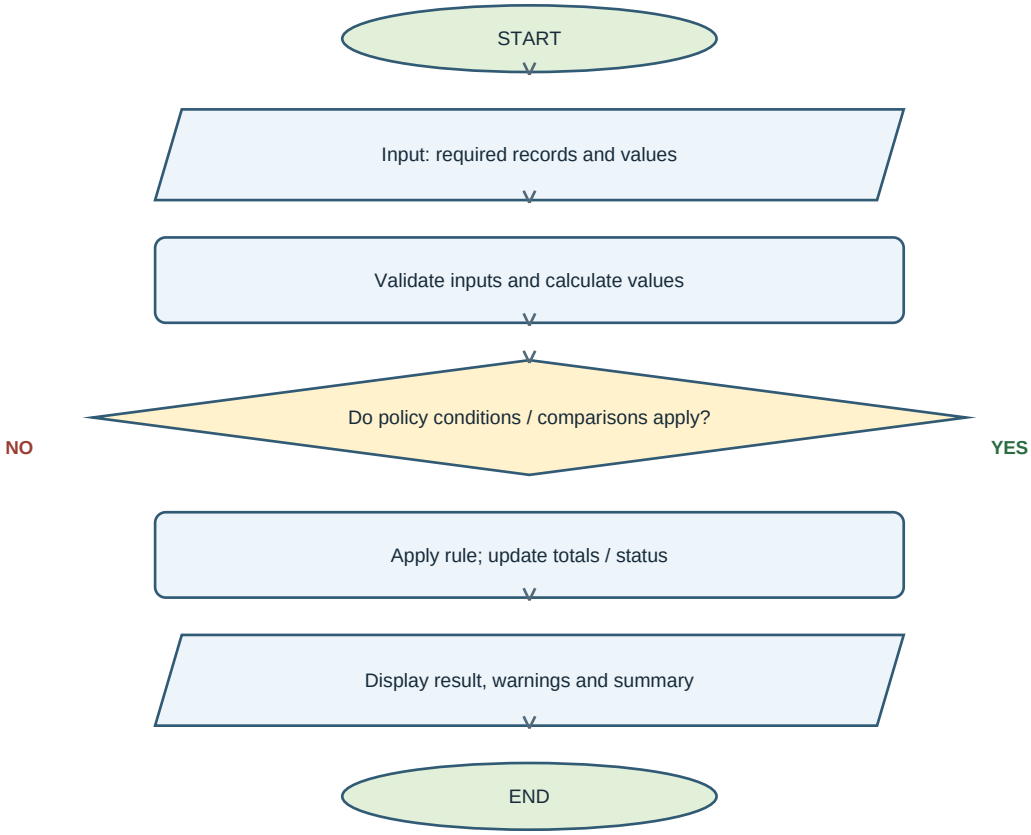
1. Read expense records and budget.

2. For each expense, convert amount to INR and add conversion fee.

3. Accumulate total and track the largest expense.

4. Compare total with budget and print the result.

5. Flowchart (standard symbols)



Symbol key: Oval = Start/End Parallelogram = Input/Output Rectangle = Process Diamond = Decision

Case Study 9: Multi-Currency Travel Budget Analysis — Implementation

6. Pseudocode

```
START
  expenses ← supplied (category, amount, rate) records
  total ← 0
  largest_category ← empty
  largest_cost ← 0
  FOR each expense
    converted ← amount * rate
    fee ← converted * 0.02
    cost ← converted + fee
    total ← total + cost
    IF cost > largest_cost
      largest_cost ← cost
      largest_category ← category
    ENDIF
    print category, converted, fee, cost
  ENDFOR
  IF total <= budget: print "Within budget"
  ELSE: print "Over budget"
  print total, largest_category
END
```

7. Python Source Code

```
expenses = [
    ("Accommodation", 200, 83.0),  # foreign amount, INR per unit
    ("Food", 100, 83.0),
    ("Transport", 80, 90.0),
    ("Sightseeing", 50, 83.0),
]
budget = 50000
total = 0
largest_category = ""
largest_cost = 0

for category, amount, rate in expenses:
    converted = amount * rate
    fee = converted * 0.02
    cost = converted + fee
    total += cost
    print(f"{category}: converted=■{converted:.2f}, fee=■{fee:.2f}, "
          f"total=■{cost:.2f}")
    if cost > largest_cost:
        largest_cost = cost
        largest_category = category

print(f"Total travel spending: ■{total:.2f}")
print("Largest expense category:", largest_category)
print("Budget status:", "Within budget" if total <= budget else "Over budget")
```

8. Test Cases

Test	Input / condition	Expected result
Accommodation 200 at rate 83	Converted ■16600; 2% fee ■332	Expense total ■16932
Total ■30000 with budget ■50000	Total does not exceed budget	Within budget
Two categories with equal maximum cost	Current logic keeps the first maximum	First maximum category is reported

9. Sample Output

```
Accommodation: converted=■16600.00, fee=■332.00, total=■16932.00
Food: converted=■8300.00, fee=■166.00, total=■8466.00
Transport: converted=■7200.00, fee=■144.00, total=■7344.00
Sightseeing: converted=■4150.00, fee=■83.00, total=■4233.00
Total travel spending: ■36975.00
Largest expense category: Accommodation
Budget status: Within budget
```

10. Conclusion

The program combines tuple records, loops, arithmetic, and maximum tracking to evaluate a travel budget. Exchange rates and fee percentages are sample inputs.

Case Study 10: Campus Event Financial Planning

1. Problem Statement

Calculate venue, equipment, catering, transport, and certificate costs; determine participant contributions, sponsorship shortfall, and minimum registration count to break even.

2. Objectives

• Calculate total event expenses and funding gap after sponsorship.

• Determine the required contribution per participant.

• Calculate the minimum number of registrations needed for break-even.

3. Assumptions / Policy Rules

• Venue, equipment, and transport are fixed costs.

• Catering and certificates are per-participant costs.

• Sponsorship is a fixed amount; participant contribution is ■500 each.

• Break-even registrations = ceiling((fixed costs - sponsorship) / (contribution - per-person variable cost)), provided the denominator is positive.

4. Algorithm

1. Read fixed costs, per-person costs, sponsorship, and contribution.

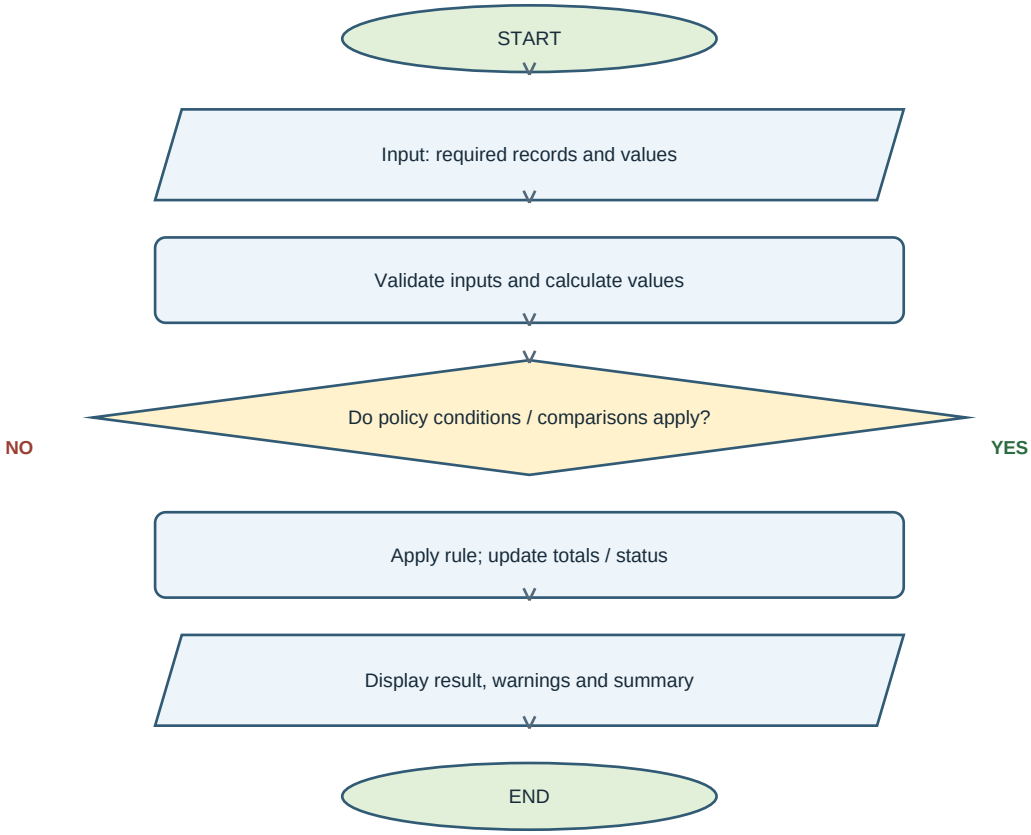
2. Calculate fixed and variable cost totals for the planned participant count.

3. Calculate total cost, total funding, and shortfall/surplus.

4. Calculate break-even registration count using the contribution margin.

5. Print financial summary and feasibility.

5. Flowchart (standard symbols)



Symbol key: Oval = Start/End Parallelogram = Input/Output Rectangle = Process Diamond = Decision

Case Study 10: Campus Event Financial Planning — Implementation

6. Pseudocode

```
START
  fixed_cost ← venue + equipment + transport
  variable_per_person ← catering + certificate
  total_cost ← fixed_cost + participants * variable_per_person
  participant_funds ← participants * contribution
  funds ← participant_funds + sponsorship
  balance ← funds - total_cost
  margin ← contribution - variable_per_person
  IF margin <= 0: print "Break-even not possible with this contribution"
  ELSE
    required ← max(0, ceiling((fixed_cost - sponsorship) / margin))
    print required
  ENDIF
  print total_cost, funds, balance
END
```

7. Python Source Code

```
import math

venue = 12000
equipment = 5000
transport = 3000
catering_per_person = 180
certificate_per_person = 20
sponsorship = 8000
contribution = 500
participants = 60

fixed_cost = venue + equipment + transport
variable_per_person = catering_per_person + certificate_per_person
total_cost = fixed_cost + participants * variable_per_person
participant_funds = participants * contribution
total_funds = participant_funds + sponsorship
balance = total_funds - total_cost
margin = contribution - variable_per_person

print(f"Fixed cost: ■{fixed_cost:.2f}")
print(f"Variable cost per participant: ■{variable_per_person:.2f}")
print(f"Total event cost: ■{total_cost:.2f}")
print(f"Participant contributions: ■{participant_funds:.2f}")
print(f"Sponsorship: ■{sponsorship:.2f}")
print(f"Funds minus cost: ■{balance:.2f}")
if margin <= 0:
    print("Break-even not possible with this contribution")
else:
    required = max(0, math.ceil((fixed_cost - sponsorship) / margin))
    print("Minimum registrations for break-even:", required)
```

8. Test Cases

Test	Input / condition	Expected result
60 participants	Fixed ■20000; variable ■200 each; total cost ■32000	Funds ■38000; surplus ■6000
Contribution equals variable cost	Contribution margin is zero	Break-even not possible using this formula
Sponsorship covers fixed cost	Fixed-cost funding gap is negative	Minimum registrations reported as zero by formula

9. Sample Output

```
Fixed cost: ■20000.00
Variable cost per participant: ■200.00
Total event cost: ■32000.00
Participant contributions: ■30000.00
Sponsorship: ■8000.00
Funds minus cost: ■6000.00
Minimum registrations for break-even: 40
```

10. Conclusion

The model separates fixed and variable costs and computes a break-even count. All amounts and contribution rules are illustrative assumptions.